

Project Information		
Group ID	13	Semester (SS2?)
1. Student Name	Taufik Yasin	SS26
2. Student Name	Musab Güler	SS26
3. Student Name	Omar Remmo	SS26
4. Student Name		
Project Title		
QuizBattle: A Distributed Multiplayer Quiz Game		
Project Description		
QuizBattle is a distributed multiplayer quiz game for a local network. Clients join a quiz, receive questions, submit answers, and see their scores. One server acts as leader and runs the active game. Other server processes act as backups and store a replicated copy of the game state. The game state includes players, scores, questions, rounds, teams, answers, sequence numbers, and previous events. The game supports individual rounds and team rounds. In team rounds, two players can exchange short team messages through the server and submit one shared team answer. Clients never communicate directly with each other. Important game events are assigned sequence numbers by the leader. Clients process these events in order using a hold-back queue and can request missing events again. If the leader fails, the remaining servers detect this through heartbeats, run a new LCR leader election, and clients rediscover the new leader.		
GitHub/GitLab/BitBucket etc. URL:		
Code Repository Link	https://github.com/fiko-source/DBQuizbattle.git https://github.com/fiko-source/DBQuizClient.git	

Architectural Model	
Architecture Type	☒ Client-Server ☐ Peer-to-Peer ☐ Hybrid
Mechanism	
Communication	Explain briefly what it is used for:
TCP	Used for reliable server-to-server control communication and for client-server WebSocket connections. Server-to-server TCP is used for LCR election messages, leader announcements, state requests, heartbeats, and state replication. TCP messages are JSON messages with a 4-byte length header. Clients use WebSockets over TCP to join the game, send answers and receive ordered game events.
UDP	Used for dynamic discovery and heartbeat announcements. Servers broadcast discovery and heartbeat messages on UDP port 5972. Clients send CLIENT_DISCOVER as UDP broadcast. Only the current leader responds with its address and WebSocket port.
Other	WebSockets are used on top of TCP for client-server communication. JSON is used as the message format. Server control messages use custom length-framed JSON over TCP.
Concurrency	Explain briefly what it is used for:
Multithreading	Used on the client side. The PyQt GUI runs in the main thread, while the asynchronous network client runs in a background thread with its own asyncio event loop. This prevents the GUI from freezing while network communication is active.
Multiprocessing	Not used. The project uses multiple independently started Python processes for servers and clients, but the code itself does not use Python 's multiprocessing module.

Text should stay within text boxes. Hidden text and text running outside text boxes will not be considered.

System Architecture Desin

Include a clear diagram of your system architecture (key components and connections between them).

Dynamic Discovery of Hosts	
Discovery Mechanism	☒ Client discovers server ☒ Server discovers servers ☐ Other. Explain briefly:
Discovery Implemented	☒ Broadcast ☐ Multicast ☐ Other. Explain briefly:
Discovery Occurs	☒ When system starts ☒ Whenever new component comes in ☒ Other. Explain briefly:
	Clients also rediscover the leader after connection loss or leader failure. Servers send periodic heartbeats to keep membership information updated.

Voting	
Voting Implemented Using	☐ Bully Algorithm ☒ LeLann-Chang-Roberts Algorithm ☐ Hirschberg-Sinclair Algorithm
Group View Used	☐ No ☒ Yes. Explain briefly what the group view is used for:
	Each server maintains a local group view of known server peers. The group view maps server UUIDs to their host, WebSocket port, control port, and last heartbeat time. It is used to form the logical ring, determine the successor, detect failed servers, start elections, and replicate state to backups.
Nodes Identified Using	☐ IP addresses / IP addresses + ports ☒ UUIDs ☐ Other (describe ID format and reasons for using it):
Election Starts	☒ When system starts ☒ When a new server joins ☒ When the leader fails

Text should stay within text boxes. Hidden text and text running outside text boxes will not be considered.

Fault Tolerance	
Faults Tolerated	☒ Crash Faults (Leader Server, Regular Server, Client) ☒ Omission Faults ☐ Byzantine Faults
Fault Detection	Explain who sends heartbeats to whom, their frequency, and retries: Servers send UDP heartbeat announcements periodically and also send confirmed TCP control heartbeats to known peers. The default heartbeat interval is 1.0 second. If a peer is not seen for more than 4.0 seconds, it is considered failed and removed from the group view. Server-to-server TCP control messages use retries. The default control timeout is 2.0 seconds and the default retry count is 3. Clients also acknowledge received event sequence numbers. If an event is missing, the client sends a RESEND_REQUEST.
Recovery Strategy	If a regular backup server fails, it is removed from the group view and the remaining servers continue operating. If the leader fails, the remaining servers detect the missing heartbeat, remove the leader from the ring, and start a new LCR election. The server with the largest remaining UUID becomes the new leader. The leader replicates the full game state to backup servers. The replicated state includes the current phase, round, current question, deadline, teams, answers, scores, processed requests, event sequence number, and event history. After leader failure, a backup that becomes leader continues from the replicated state. Clients recover by rediscovering the leader. When a client reconnects, it sends its token and last received sequence number. The new leader sends missing events so the client can continue with the current game state.

*Text should stay within text boxes. Hidden text and text running outside text boxes will not be considered.
 Make sure the final file is a non-editable PDF (save or export as a PDF).*